

Lesson 01 Demo 04

Performing DAST Scan on a Running Web Application with OWASP ZAP

Objective: To configure and execute a DAST scan on a running web application using OWASP ZAP integrated with GitHub Actions for identifying vulnerabilities and ensuring application security

Tools required: GitHub Actions and OWASP ZAP DAST tool

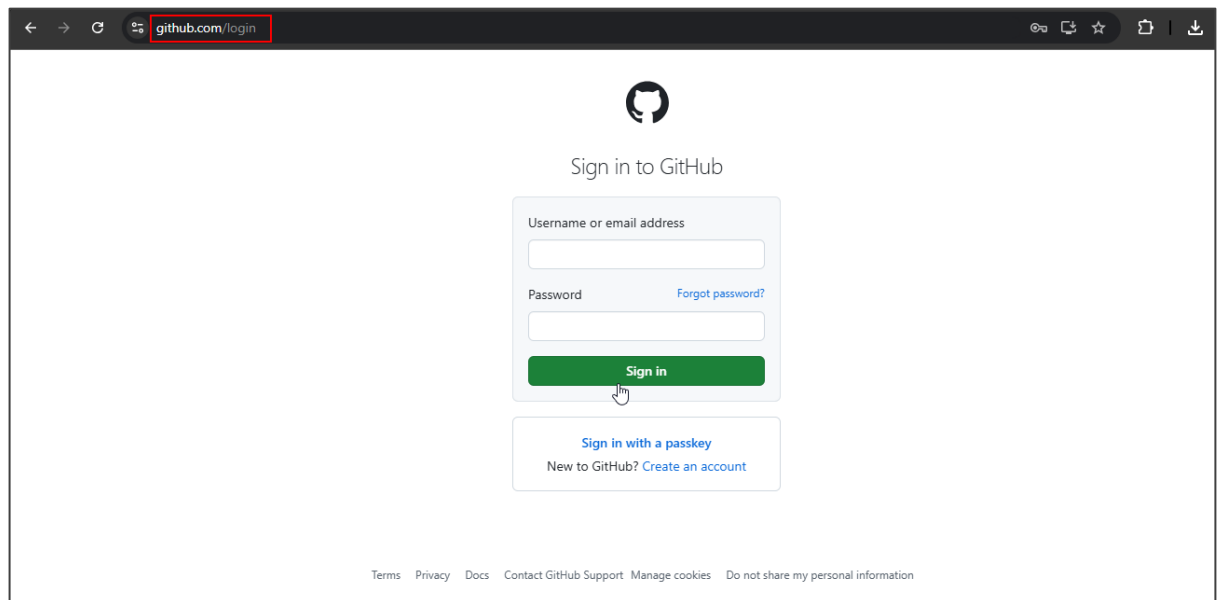
Prerequisites: None

Steps to be followed:

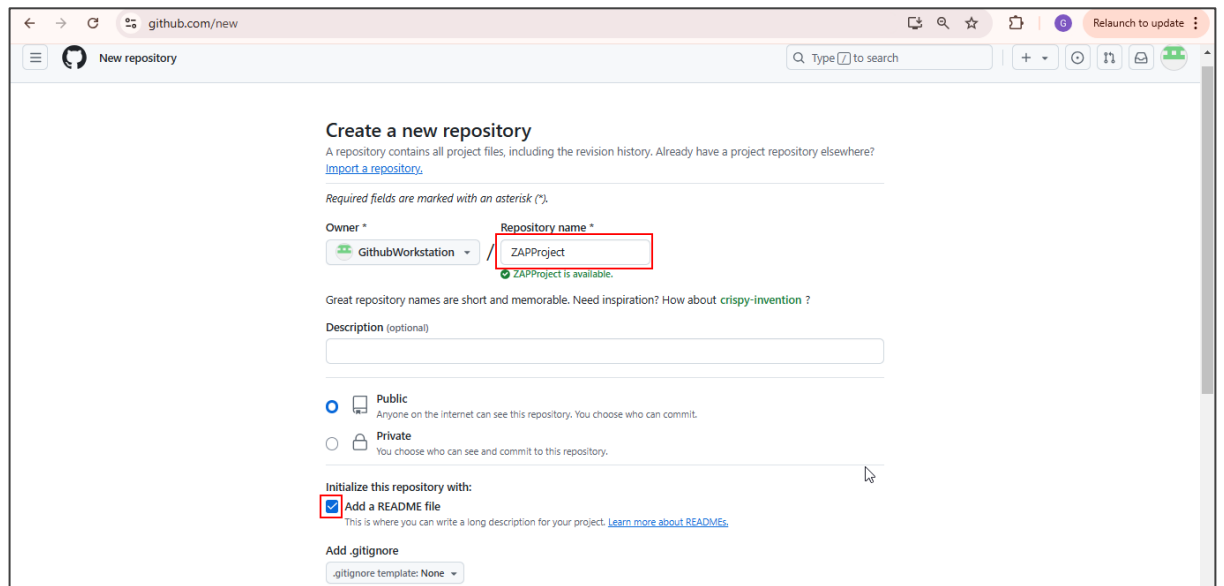
1. Configure DAST tool with GitHub Actions

Step 1: Configure DAST tool with GitHub Actions

- 1.1 Sign in to your GitHub account using the following URL:
<https://github.com/login>



1.2 Create a repository, add a desired name, and click on the check box to add a README file



github.com/new

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

Required fields are marked with an asterisk (*).

Owner * GithubWorkstation / Repository name * ZAPProject
✓ ZAPProject is available.

Great repository names are short and memorable. Need inspiration? How about [crispy-invention](#) ?

Description (optional)

☒ Public
Anyone on the internet can see this repository. You choose who can commit.

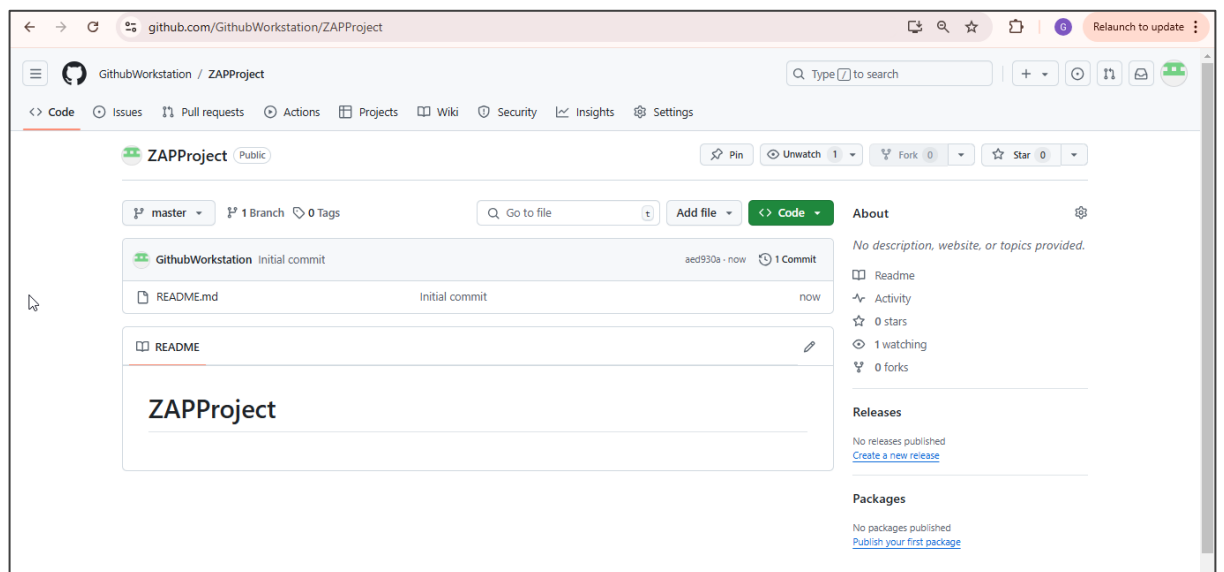
☐ Private
You choose who can see and commit to this repository.

Initialize this repository with:

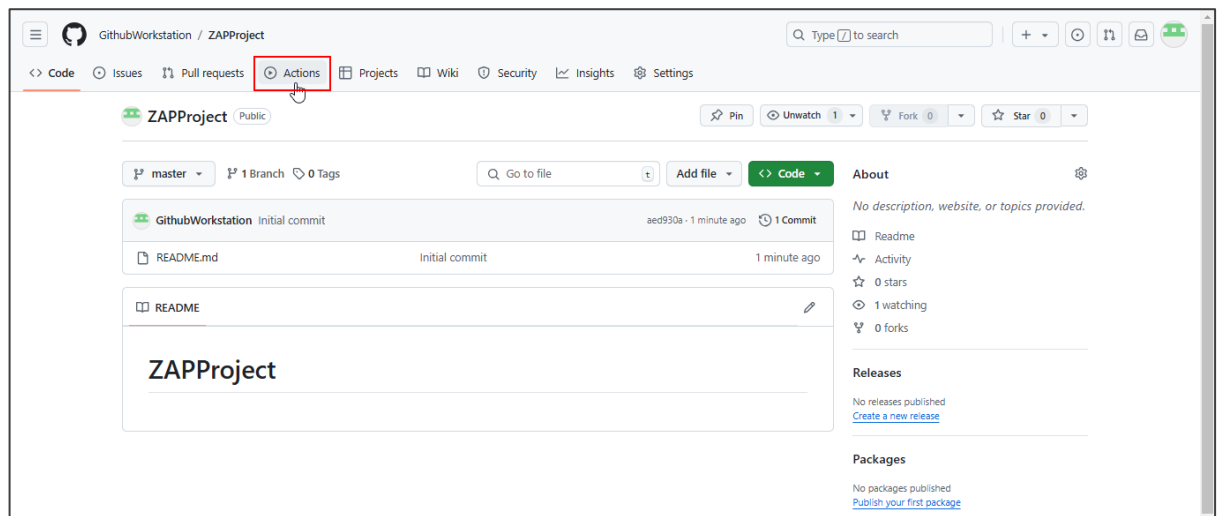
☒ Add a README file
This is where you can write a long description for your project. [Learn more about READMEs.](#)

Add .gitignore
.gitignore template: None

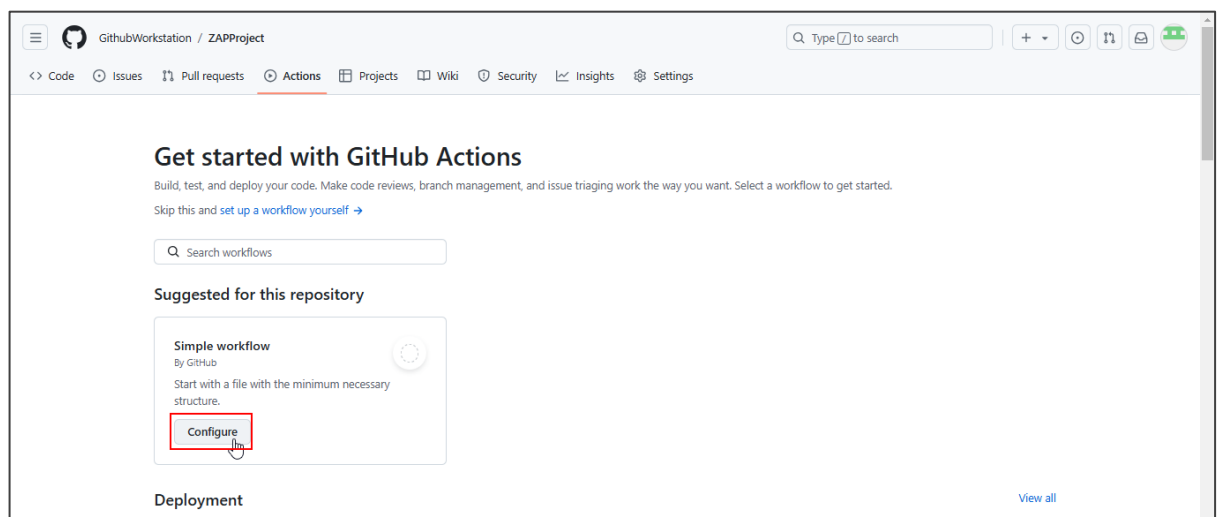
The repository is created successfully as shown below:



1.3 Click on the **Actions** tab



1.4 Click on **Configure** in the **Simple workflow** section



1.5 Add the following code to the **.yml** file as shown below:

name: OWASP ZAP Scan

on:

pull_request:

branches:

- main # Trigger ZAP scan on PRs targeting the `main` branch

push:

branches:

- main # Or trigger on push to the `main` branch

workflow_dispatch: # Allow manual triggering of the workflow

permissions:

issues: write

jobs:

zap_scan:

runs-on: ubuntu-latest

steps:

Checkout the repository

- name: Checkout Repository

uses: actions/checkout@v2

Set up Python (optional, if your application is Python-based)

- name: Set up Python

uses: actions/setup-python@v2

with:

python-version: '3.8'

Run ZAP Scan

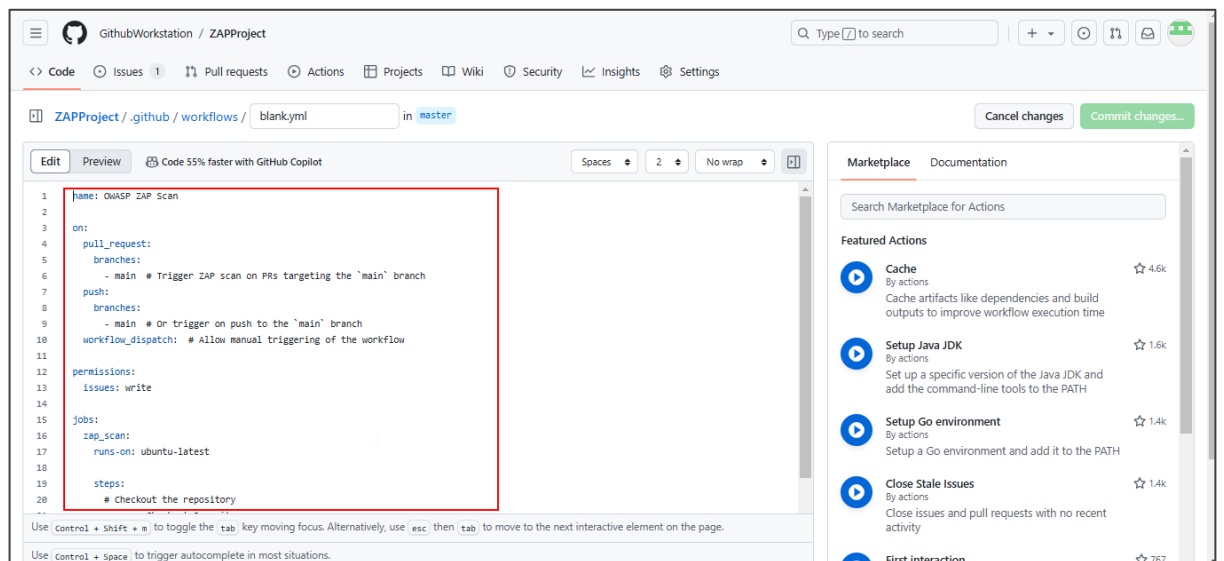
- name: ZAP Scan

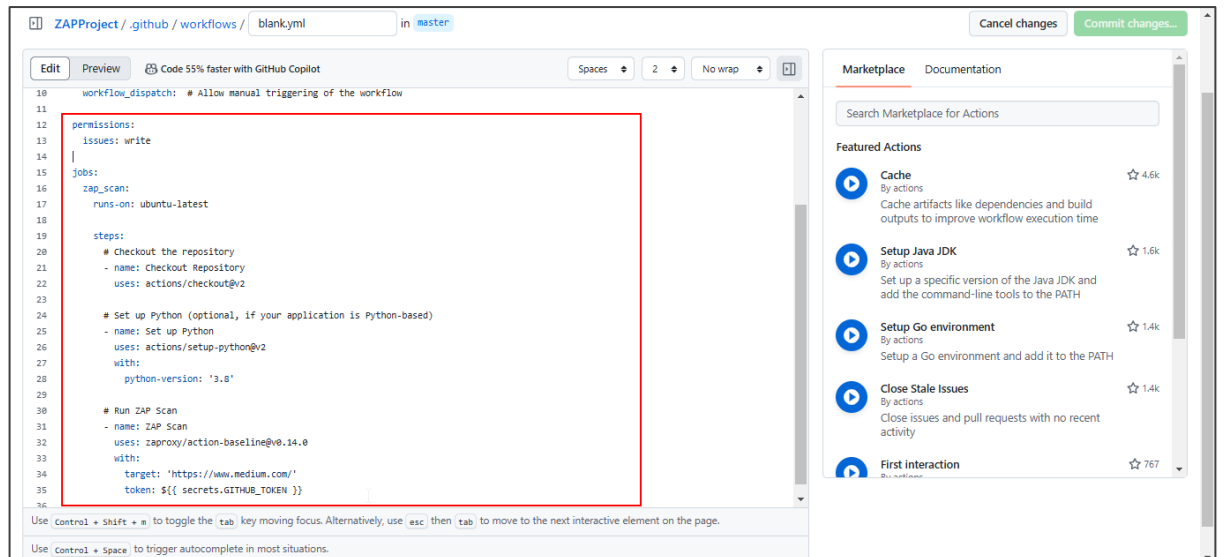
uses: zapproxy/action-baseline@v0.14.0

with:

target: 'https://www.medium.com/'

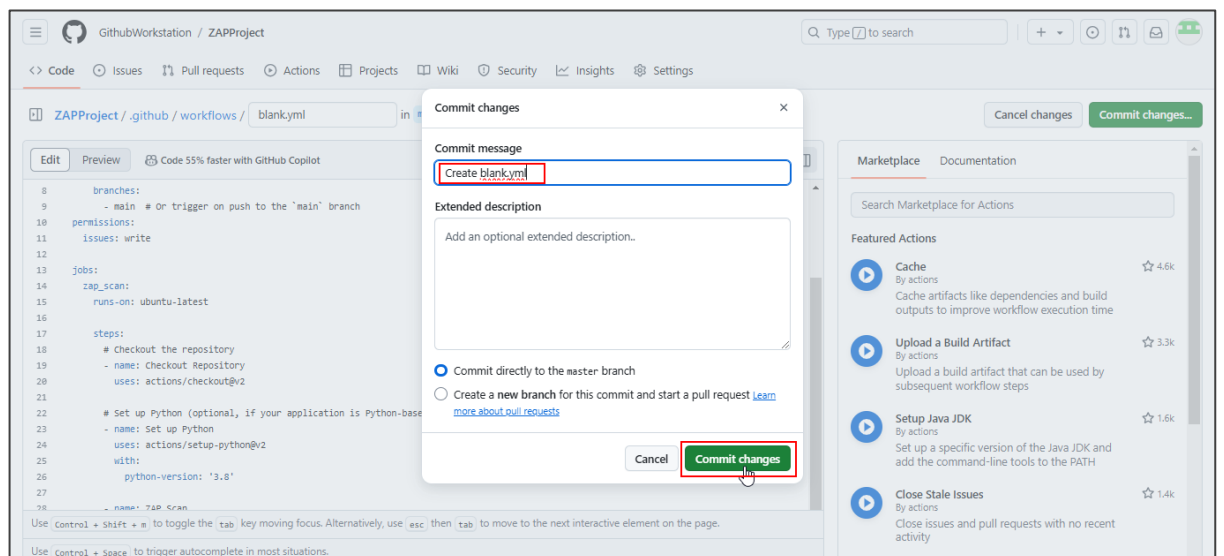
token: \${{ secrets.GITHUB_TOKEN }}



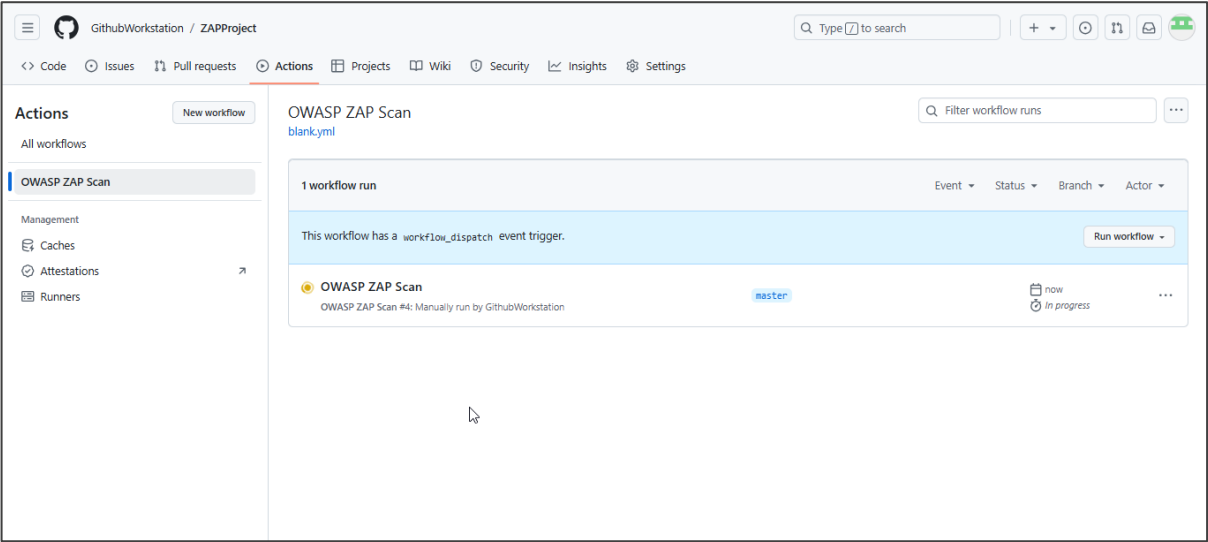


Note: The ZAP Baseline Python script spiders the target URL for one minute and provides scan results for the specified website.

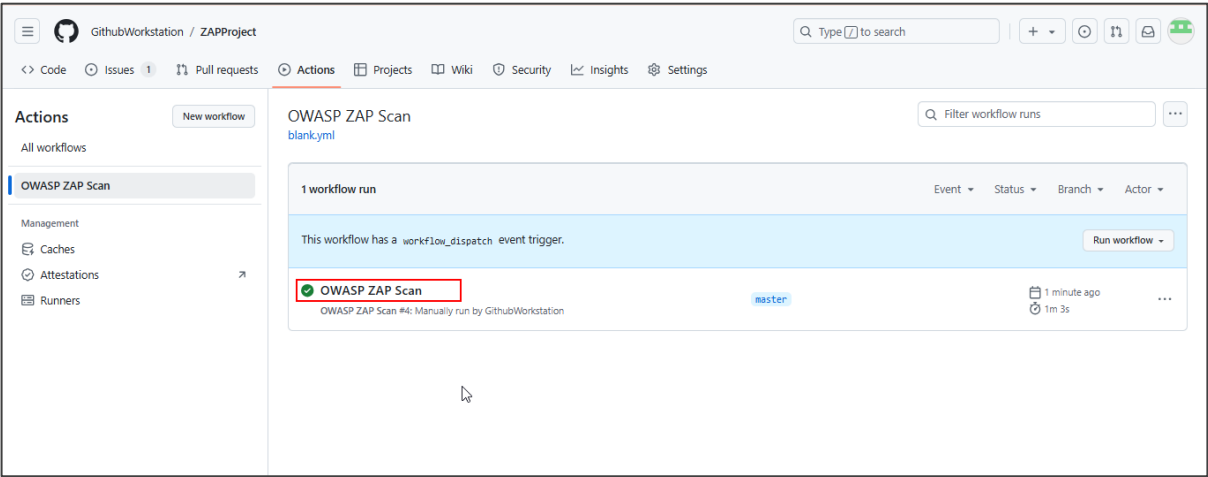
1.6 Click on the **Commit changes...** button, add a commit message, and then click **Commit changes**



1.7 Navigate to the **Actions** tab to view the workflow



1.8 Click on **OWASP ZAP Scan** as shown below:



1.9 Navigate to **zap_scan** in the **Jobs** section and view the detailed steps as shown below:

The screenshot shows the GitHub Actions interface for the 'OWASP ZAP Scan' workflow. The 'Jobs' section on the left lists 'zap_scan' as the active job. The main panel displays the 'zap_scan' job details, which succeeded 2 minutes ago. The job steps are: 'Set up job' (15s), 'Checkout Repository' (15s), 'Set up Python' (0s), and 'ZAP Scan' (51s). The 'ZAP Scan' step is expanded, showing the following log output:

```
1 ▶ Run zaproxy/action-baseline@v0.14.0
12 starting the program
13 github run id :12023286553
14 /usr/bin/cmod a-w /home/runner/work/ZAPProject/ZAPProject
15 /usr/bin/docker pull ghcr.io/zaproxy/zaproxy:stable -q
16 ghcr.io/zaproxy/zaproxy:stable
17 /usr/bin/docker run -v /home/runner/work/ZAPProject/ZAPProject:/zap/work --network=host -e ZAP_AUTH_HEADER -e ZAP_AUTH_HEADER_VALUE -e ZAP_AUTH_HEADER_NAME -e ghcr.io/zaproxy/zaproxy:stable zap-baseline.py -t https://www.medium.com/ -c report_json.json -w report_md.md -r report_html.html
18 Using the Automation Framework
19 Total of 4 URLs
20 PASS: Vulnerable JS Library (Powered by Retire.js) [10003]
21 PASS: In Page Banner Information Leak [10009]
22 PASS: Cookie No HttpOnly Flag [10010]
23 PASS: Cookie Without Secure Flag [10011]
24 PASS: Re-examine Cache-control Directives [10015]
```

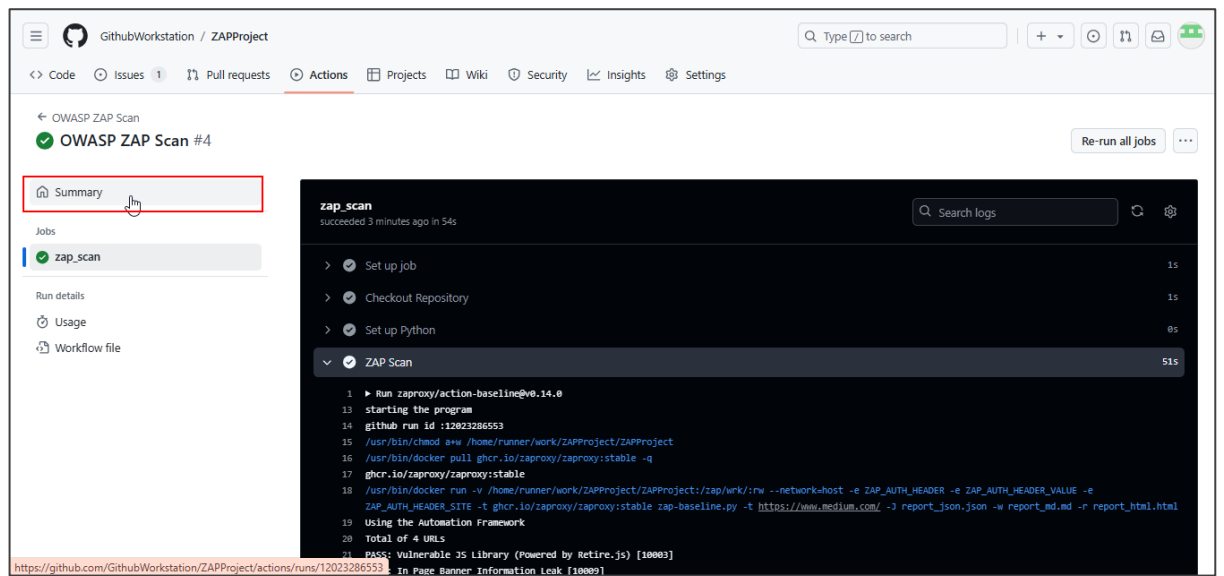
The screenshot shows the GitHub Actions interface for the 'OWASP ZAP Scan' workflow. The 'Jobs' section on the left lists 'zap_scan' as the active job. The main panel displays the 'zap_scan' job details, which succeeded 3 minutes ago. The job steps are: 'ZAP Scan' (51s), 'Post Set up Python' (0s), and 'Post Checkout Repository' (0s). The 'ZAP Scan' step is expanded, showing the following log output:

```
118 Process completed successfully and a new issue #1 has been created for the ZAP Scan.
119 Artifact name is valid!
120 Root directory input is valid!
121 Beginning upload of artifact content to blob storage
122 Uploaded bytes 29775
123 Finished uploading artifact content to blob storage!
124 SHA256 hash of uploaded artifact zip is 8c230867d461748c4d1d4a900b4ca076fef6d815b606e1dfb1f9e2c363f13042
125 Finalizing artifact upload
126 Artifact zap_scan.zip successfully finalized. Artifact ID 2236647488
```

The 'Post Set up Python' step is expanded, showing the following log output:

```
1 Post job cleanup.
2 /usr/bin/git version
3 git version 2.47.0
4 Temporarily overriding HOME="/home/runner/work/_temp/80955738-b25a-4cac-a944-107163cedce8" before making global git config changes
5 Adding repository directory to the temporary git global config as a safe directory
6 /usr/bin/git config --global --add safe.directory /home/runner/work/ZAPProject/ZAPProject
7 /usr/bin/git config --local --name-only --get-regexp core.sshCommand
8 /usr/bin/git submodule foreach --recursive sh -c "git config --local --name-only --get-regexp 'core.sshCommand' && git config --local --unset-all 'core.sshCommand' || :"
9 /usr/bin/git config --local --name-only --get-regexp http.https://github.com/\./extraheader
10 http.https://github.com/\./extraheader
11 /usr/bin/git config --local --unset-all http.https://github.com/\./extraheader
12 /usr/bin/git submodule foreach --recursive sh -c "git config --local --name-only --get-regexp 'http.https://github.com/\./extraheader' && git config
```

1.10 Click on **Summary**



GitHubWorkstation / ZAPProject

OWASP ZAP Scan

OWASP ZAP Scan #4

Summary

Jobs

zap_scan

Run details

Usage

Workflow file

zap_scan

succeeded 3 minutes ago in 54s

Search logs

Set up job 1s

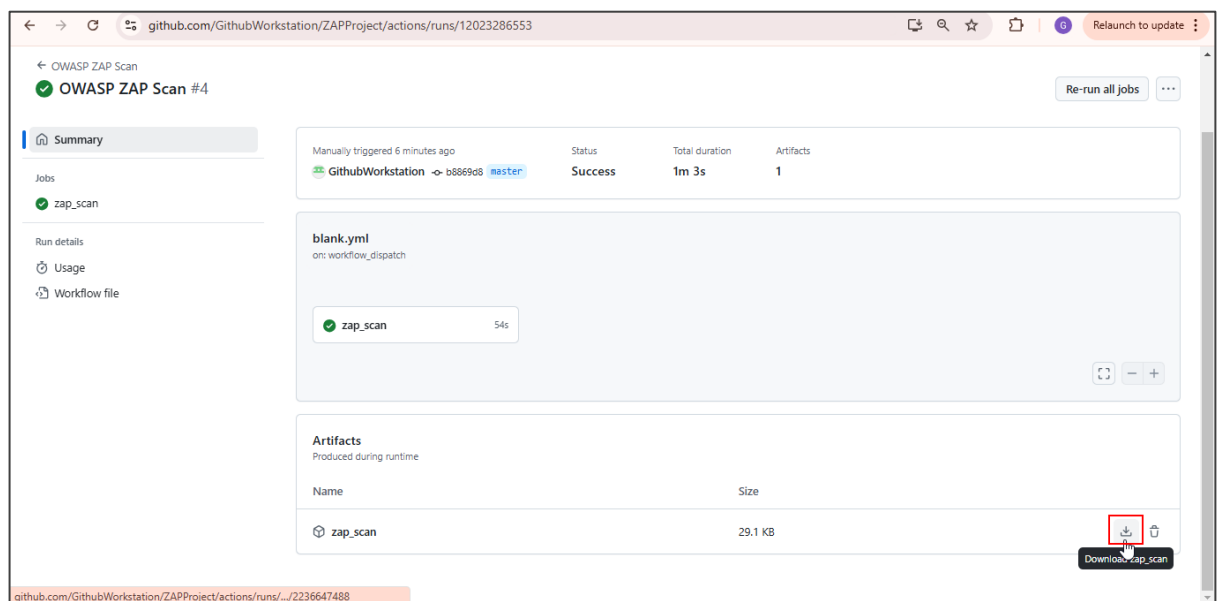
Checkout Repository 1s

Set up Python 0s

ZAP Scan 51s

```
1 ▶ Run zapproxy/action-baseline@v0.14.0
13 starting the program
14 github run id :12023286553
15 /usr/bin/chmod a+w /home/runner/work/ZAPProject/ZAPProject
16 /usr/bin/docker pull ghcr.io/zaproxy/zaproxy:stable -q
17 ghcr.io/zaproxy/zaproxy:stable
18 /usr/bin/docker run -v /home/runner/work/ZAPProject/ZAPProject:/zap/work --network-host -e ZAP_AUTH_HEADER -e ZAP_AUTH_HEADER_VALUE -e
19 ZAP_AUTH_HEADER_SITE -t ghcr.io/zaproxy/zaproxy:stable zap-baseline.py -t https://www.medium.com/ -j report_json.json -w report_md.md -r report_html.html
20 Using the Automation Framework
21 Total of 4 UMLs
22 PASS: Vulnerable JS Library (Powered by Retire.js) [10003]
https://github.com/GithubWorkstation/ZAPProject/actions/runs/12023286553 In Page Banner Information Leak [10003]
```

1.11 Click on the download icon in the **Artifacts** section as shown below:



github.com/GithubWorkstation/ZAPProject/actions/runs/12023286553

OWASP ZAP Scan

OWASP ZAP Scan #4

Summary

Jobs

zap_scan

Run details

Usage

Workflow file

Manually triggered 6 minutes ago

GitHubWorkstation → b866908 master

Status Success

Total duration 1m 3s

Artifacts 1

blank.yml

on: workflow_dispatch

zap_scan 54s

Artifacts

Produced during runtime

Name	Size
zap_scan	29.1 KB

Download zap_scan

1.12 Open the downloaded **report_html.html** file to view the ZAP report as shown below:

File /AppData/Local/Temp/86550bad-e913-4f2d-af96-948a0a6f7d69_zap_scan.zip.d69/report_html.html



ZAP Scanning Report

Site: <https://www.medium.com>
Generated on Tue, 26 Nov 2024 03:33:46
ZAP Version: 2.15.0
ZAP by [Checkmarx](#)

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	4
Low	3
Informational	6
False Positives:	0

Alerts

Name	Risk Level	Number of Instances
CSP: Wildcard Directive	Medium	1
CSP: script-src unsafe-inline	Medium	1
CSP: style-src unsafe-inline	Medium	1

Alerts

Name	Risk Level	Number of Instances
CSP: Wildcard Directive	Medium	1
CSP: script-src unsafe-inline	Medium	1
CSP: style-src unsafe-inline	Medium	1
Sub Resource Integrity Attribute Missing	Medium	15
Cookie with SameSite Attribute None	Low	2
Cross-Domain JavaScript Source File Inclusion	Low	6
Permissions Policy Header Not Set	Low	1
Information Disclosure - Suspicious Comments	Informational	2
Loosely Scoped Cookie	Informational	2
Non-Storable Content	Informational	1
Retrieved from Cache	Informational	1
Session Management Response Identified	Informational	3
Storable and Cacheable Content	Informational	3

Alert Detail

Medium	CSP: Wildcard Directive
Description	Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks. Including (but not limited to) Cross Site Scripting (XSS), and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page — covered types are JavaScript, CSS, HTML frames, fonts, images and embeddable objects such as Java applets, ActiveX, audio and video files.
URL	https://www.medium.com/
Method	GET
Parameter	content-security-policy

By following these steps, you have successfully configured and executed a DAST scan on a running web application using OWASP ZAP to identify vulnerabilities and enhance security.